

Estado del proyecto — Robot Bípedo (resumen para el equipo)

Resumen de dónde estamos, qué funciona, y qué sigue. Para que todos estemos en la misma página.

1. El proyecto en una frase

Robot bípedo interactivo para promociones comerciales. Tiene **dos grandes partes**: - **Sistema interactivo** (cámara, QR, cédula, factura, base de datos) → **FUNCIONANDO**. - **Robot autoequilibrado** (que se mantiene erguido solo sobre 2 ruedas) → **en construcción**.

2. Primer avance — FUNCIONANDO 🟢

El sistema interactivo completo ya anda de punta a punta (probado en vivo: venta registrada, aparece en el dashboard y en Firebase).

Arquitectura:

```
Celular (cámara IP) → Raspberry Pi → PC (visión + base de datos)
Pi → ESP32 → VESC → ruedas
```

Qué hace cada parte: - **PC:** visión por computadora (MediaPipe para manos, OpenCV para QR) y base de datos. Dos programas: `pc_server.py` (visión, puerto 5555) y `api_server.py` (dashboard web + registro de ventas, puerto 8000). - **Raspberry Pi:** interfaz gráfica (Pygame), botones físicos, validación de cédula (Módulo 10), y coordina todo. Archivos: `pi_gui_gpio.py`, `pi_zmq_client.py`, `pi_uart.py`. - **ESP32:** control de movimiento. Habla con la VESC por protocolo binario. - **Base de datos:** SQLite local + Firebase (nube), escritura doble. Si no hay internet, guarda local y sincroniza después.

Flujo de una interacción (lo que hace hoy): Modo atracción (ruedas rodando) → detecta la mano → gira y frena → pide el QR → muestra la oferta → ingresás la cédula con botones → factura → registra la venta (local + nube) → "muchas gracias" → vuelve a rodar. Si pasan 20 s sin QR, agradece y sigue (no se trava).

3. Hardware actual

- Raspberry Pi 4 + pantalla.
- Celular como cámara IP (app **IP Webcam** por WiFi).
- **ESP32** (se programa con PlatformIO, framework ESP-IDF — C nativo, no Arduino).

- **VESC dual Autoro AESC DV6.7** — dos controladores en una placa, unidos por CAN. IDs: **maestra = 42**, **esclava = 25**.
 - 2 motores hub (uno por rueda).
 - **Conexiones del ESP32:** GPIO16/17 ↔ UART de la Pi; GPIO4/5 ↔ UART de la VESC; GND común entre Pi, ESP32 y VESC.
-

4. Lecciones aprendidas (importantes, para no repetir errores)

- **Las IPs cambian en cada red WiFi.** Al cambiar de lugar hay que actualizar `pc_ip` y `camara_url` en `config.json`. La red de la U es inestable; un **hotspot** de celular da una red más estable.
 - **El GND común (Pi ↔ ESP32 ↔ VESC) es crítico.** La mayoría de los problemas de comunicación que tuvimos eran cables flojos o cortos en la protoboard. Siempre medir continuidad y que las masas estén juntas.
 - **El código nunca falló** en la presentación — los problemas fueron de cableado.
 - Firebase necesita internet; sin internet el robot funciona igual (guarda local).
-

5. Fase nueva — Robot autoequilibrado (en construcción)

Estamos empezando a construir el robot que se mantiene parado solo (tipo Segway, dos ruedas). **Decisiones clave ya tomadas:**

- **Camino B: el balanceo lo hace el ESP32.** El ESP32 lee la IMU (MPU6050), corre el PID de equilibrio y le manda **corriente** a las dos ruedas. No usamos la app Balance de la VESC (está pensada para vehículos con una persona encima; la nuestra es autónoma). Este camino reusa el protocolo ESP32↔VESC que ya tenemos.
 - **Piernas rectas (180°), no con rodilla** — más rígidas = mejor para balancear.
 - **Altura máxima ~1.20 m.**
 - La MPU6050 se conecta al **ESP32** (I2C, GPIO21/22).
 - El plan completo está en `ROADMAP_BALANCEO.md` — 7 etapas, cada una probada con las ruedas en el aire antes de avanzar.
-

6. Dónde estamos AHORA mismo

Etapas 1 del balanceo: validar que se pueda mandar **corriente** a las dos ruedas. - Paso inmediato: probar en **VESC Tool** que ambas ruedas giran en modo corriente. - Después: firmware del ESP32 que manda `SET_CURRENT` a las dos ruedas.

Regla de oro del balanceo: todo se prueba con las **ruedas en el aire** y con el **kill switch a mano**, hasta estar seguros.

7. Próximos pasos (etapas del balanceo)

1. **Etapa 1:** corriente a las 2 ruedas desde el ESP32. ← *acá estamos*
 2. **Etapa 2:** leer el ángulo (pitch) de la MPU6050 con filtro complementario.
 3. **Etapa 3:** chequear el **signo** del lazo (en el aire — crítico).
 4. **Etapa 4:** primer balanceo real (con alguien sosteniendo).
 5. **Etapa 5:** navegación (avanzar / girar).
 6. **Etapa 6:** integrar con la Pi y el sistema interactivo.
 7. **Etapa 7:** seguridad y ajuste fino.
-

8. Documentos clave en el repositorio

- **ROADMAP_BALANCEO.md** — el plan del robot autoequilibrado (lo que estamos haciendo).
 - **DOCUMENTACION_PROYECTO.md** — documentación técnica completa (para la exposición).
 - **GUIA_ARRANQUE.md** — cómo prender y hacer funcionar todo el sistema.
 - Carpetas **pc/** , **pi/** , **esp32/** — el código de cada parte.
-

9. Pendientes / decisiones abiertas

- Confirmar el **sentido de giro** de las dos ruedas (para que vaya derecho).
 - Cablear los **2 sensores IR** de obstáculos (el código ya está listo, desactivado).
 - Definir si hace falta una **consulta propia** de los datos en la nube (por ahora alcanza con el dashboard local y la consola de Firebase).
-

Repo: github.com/FernandoVB01/Robot_Bipedo